Green River Mammoth Cave USGS Gauge Data (Site 03309000) Anlysis of flood events since removal of L&D6 (2018-2026)

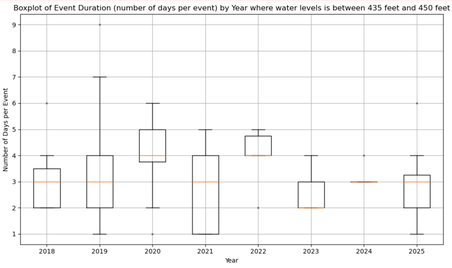
```
# Install dependencies
!pip install -q pandas
!pip install -q numpy
!pip install -q matplotlib
!pip install -q seaborn
!pip install -q requests
!pip install -q scipy
Requirement already satisfied: python-dateutil>=2.8.2,<2.9.0dev,>=2.7.0 from /usr/local/lib/python3.12/dist-packages/botocore-1.35.99,>=1.37.4,<1.38.0dev,>=1.35.0,>=1.37.1,<1.38.0dev,>=1.35.0; extra == "boto3" in /usr/local/lib/python3.12/dist-packages/boto3-1.35.99,>=1.37.4,<1.38.0,>=1.35.0,>=1.37.1,<1.38.0,>=1.35.0; extra == "boto3" in /usr/local/lib/python3.12/dist-packages/s3transfer-0.10.4,>=0.10.0,<0.11.0 in /usr/local/lib/python3.12/dist-packages/s3transfer-0.10.4,>=0.10.0,<0.11.0; extra == "boto3" in /usr/local/lib/python3.12/dist-packages/s3transfer-0.10.4,>=0.10.0,<0.11.0; extra == "boto3" in /usr/local/lib/python3.12/dist-packages/urllib3-2.3.0,<1.26,>=1.25.4; extra == "botocore
```

In [116…

```
# Ensure elevation column exists, then filter for between 435 ft and 450 feet the 100 year flood level
if 'max_daily_elev_ft' not in daily_max_df.columns:
    daily_max_df['max_daily_elev_ft'] = daily_max_df['max_daily_gage'] + offset_ft

df_435 = daily_max_df[daily_max_df['max_daily_elev_ft'].between(435,450)].reset_index(drop=True)


# Sort by date to ensure proper sequence detection
df_435_sorted = df_435.sort_values('date').reset_index(drop=True)

# Calculate the difference between consecutive dates
df_435_sorted['date_diff'] = df_435_sorted['date'].diff().dt.days

# Identify event breaks (where date difference > 1 day)
df_435_sorted['event_id'] = (df_435_sorted['date_diff'] > 1).cumsum()

# Count events and days per event
event_summary_435 = df_435_sorted.groupby('event_id').agg(
    event_start=('date', 'min'),
    event_end=('date', 'max'),
    num_days=('date', 'count'),
    max_elevation=('max_daily_elev_ft', 'max')
).reset_index(drop=True)

# Renumber events starting from 1
event_summary_435.insert(0, 'event_num', range(1, len(event_summary_435) + 1))

print(f"Total number of events where water levels is between 435 feet and 450 feet (below the 100 year flood mark): {len(event_summary_435)}")
print(f"\n{event_summary_435}")

# add a year column so we can group by it
event_summary_by_year_435 = event_summary_435.copy()
event_summary_by_year_435['year'] = pd.to_datetime(event_summary_by_year_435['event_start']).dt.year

# group the event durations by year and make a boxplot
grouped_year = event_summary_by_year_435.groupby('year')['num_days']

plt.figure(figsize=(10, 6))
plt.boxplot(
    [grp for _, grp in grouped_year],
    labels=grouped_year.groups.keys(),
    flierprops={'marker': 'x', 'markerfacecolor': 'red',
                'markeredgecolor': 'black', 'markersize': 3, 'alpha': 0.5}
)
plt.title('Boxplot of Event Duration (number of days per event) by Year where water levels is between 435 feet and 450 feet')
plt.xlabel('Year')
plt.ylabel('Number of Days per Event')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Total number of events where water levels is between 435 feet and 450 feet (below the 100 year flood mark): 68

    event_num event_start  event_end  num_days  max_elevation
0           1  2018-02-08  2018-02-09         2         436.71
1           2  2018-02-11  2018-02-13         3         443.95
2           3  2018-02-18  2018-02-19         2         437.62
3           4  2018-02-22  2018-02-27         6         442.97
4           5  2018-03-25  2018-03-26         2         437.44
..        ...         ...         ...       ...            ...
63         64  2025-04-10  2025-04-11         2         446.85
64         65  2025-05-04  2025-05-06         3         438.80
65         66  2025-05-17  2025-05-19         3         443.16
66         67  2025-05-21  2025-05-23         3         441.73
67         68  2025-06-17  2025-06-22         6         441.51

[68 rows x 5 columns]
```

Boxplot of Event Duration (number of days per event) by Year where water levels is between 435 feet and 450 feet



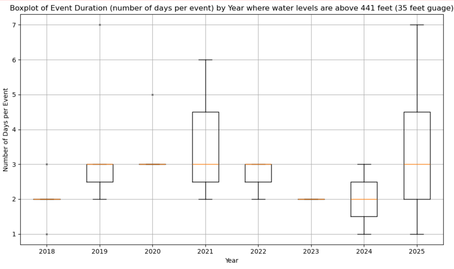
Evaluate the number of flood events with the lenght of the event and max river height during event.

In [117…

```
# Box plot of days were river was above 441 ft, the elevation where gauge is 35 feet.
# Ensure elevation column exists, then filter for >= 441 feet
if 'max_daily_elev_ft' not in daily_max_df.columns:
    daily_max_df['max_daily_elev_ft'] = daily_max_df['max_daily_gage'] + offset_ft

df_441 = daily_max_df[daily_max_df['max_daily_elev_ft']>= 441].reset_index(drop=True)

# Check if there is any data to display
if df_441.empty:
    print("No data available: Water elevation never reached 441 feet in this dataset.")
else:
    # Sort by date to ensure proper sequence detection
    df_441_sorted = df_441.sort_values('date').reset_index(drop=True)

    # Calculate the difference between consecutive dates
    df_441_sorted['date_diff'] = df_441_sorted['date'].diff().dt.days

    # Identify event breaks (where date difference > 1 day)
    df_441_sorted['event_id'] = (df_441_sorted['date_diff'] > 1).cumsum()

    # Count events and days per event
    event_summary_441 = df_441_sorted.groupby('event_id').agg(
        event_start=('date', 'min'),
        event_end=('date', 'max'),
        num_days=('date', 'count'),
        max_elevation=('max_daily_elev_ft', 'max')
    ).reset_index(drop=True)

    # Renumber events starting from 1
    event_summary_441.insert(0, 'event_num', range(1, len(event_summary_441) + 1))

    print(f"Total number of events where water levels is above 441 feet: {len(event_summary_441)}")
    print(f"\n{event_summary_441}")

    # add a year column so we can group by it
    event_summary_by_year_441 = event_summary_441.copy()
    event_summary_by_year_441['year'] = pd.to_datetime(event_summary_by_year_441['event_start']).dt.year

    # group the event durations by year and make a boxplot
    grouped_year = event_summary_by_year_441.groupby('year')['num_days']

    plt.figure(figsize=(10, 6))
    plt.boxplot(
        [grp for _, grp in grouped_year],
        labels=grouped_year.groups.keys(),
        flierprops={'marker': 'x', 'markerfacecolor': 'red',
                    'markeredgecolor': 'black', 'markersize': 3, 'alpha': 0.5}
    )
    plt.title('Boxplot of Event Duration (number of days per event) by Year where water levels are above 441 feet (35 feet guage)')
    plt.xlabel('Year')
    plt.ylabel('Number of Days per Event')
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

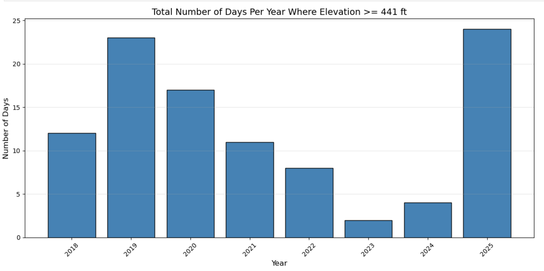
```
Total number of events where water levels is above 441 feet: 34

    event_num event_start  event_end  num_days  max_elevation
0           1  2018-02-11  2018-02-13         3         443.95
1           2  2018-02-22  2018-02-23         2         442.97
2           3  2018-02-25  2018-02-26         2         441.73
3           4  2018-03-30  2018-03-31         2         443.23
4           5  2018-11-16  2018-11-16         1         441.09
5           6  2018-12-02  2018-12-03         2         442.04
6           7  2019-01-01  2019-01-02         2         443.09
7           8  2019-02-08  2019-02-09         2         443.48
8           9  2019-02-12  2019-02-14         3         446.42
9          10  2019-02-21  2019-02-27         7         451.97
10         11  2019-12-01  2019-12-03         3         446.60
11         12  2019-12-17  2019-12-19         3         447.39
12         13  2019-12-30  2020-01-01         3         444.74
13         14  2020-02-05  2020-02-07         3         445.54
14         15  2020-02-11  2020-02-15         5         444.90
15         16  2020-03-03  2020-03-05         3         446.57
16         17  2020-03-13  2020-03-15         3         443.85
17         18  2020-03-20  2020-03-22         3         444.83
18         19  2021-01-26  2021-01-28         3         447.64
19         20  2021-02-28  2021-03-05         6         460.29
20         21  2021-05-05  2021-05-06         2         445.77
21         22  2022-01-02  2022-01-04         3         449.35
22         23  2022-01-10  2022-01-11         2         444.35
23         24  2022-02-04  2022-02-06         3         445.66
24         25  2023-02-17  2023-02-18         2         444.45
25         26  2024-05-09  2024-05-11         3         445.70
26         27  2024-05-28  2024-05-28         1         442.36
27         28  2025-01-31  2025-02-02         3         445.53
28         29  2025-02-15  2025-02-20         6         461.81
29         30  2025-03-16  2025-03-18         3         447.22
30         31  2025-04-04  2025-04-10         7         469.01
31         32  2025-05-17  2025-05-18         2         443.16
32         33  2025-05-22  2025-05-22         1         441.73
33         34  2025-06-19  2025-06-20         2         441.51
```

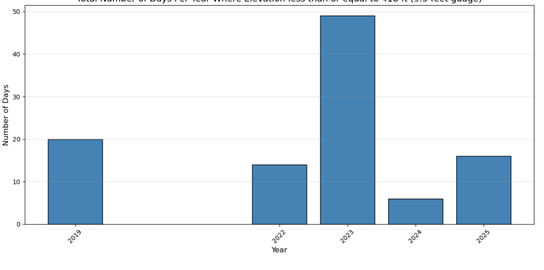
Boxplot of Event Duration (number of days per event) by Year where water levels are above 441 feet (35 feet guage)



In [114…

```
# Create a simple bar chart of the number of days per year where daily max elevation >= 441 ft
days_per_year_441 = event_summary_by_year_441.groupby('year')['num_days'].sum().reset_index()
days_per_year_441.columns = ['year', 'total_days']

plt.figure(figsize=(12, 6))
plt.bar(days_per_year_441['year'], days_per_year_441['total_days'], color='steelblue', edgecolor='black')
plt.title('Total Number of Days Per Year Where Elevation >= 441 ft', fontsize=14)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Number of Days', fontsize=12)
plt.xticks(days_per_year_441['year'], rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



In [120…

```
# Create a simple bar chart of the number of days per year where daily max elevation <= 418 ft
days_per_year_418 = daily_max_df[daily_max_df['max_daily_elev_ft'] <= 418].copy()
days_per_year_418['year'] = pd.to_datetime(days_per_year_418['date']).dt.year
days_count_418 = days_per_year_418.groupby('year').size().reset_index(name='total_days')

plt.figure(figsize=(12, 6))
plt.bar(days_count_418['year'], days_count_418['total_days'], color='steelblue', edgecolor='black')
plt.title('Total Number of Days Per Year Where Elevation less than or equal to 418 ft (9.5 feet guage)', fontsize=14)
plt.xlabel('Year', fontsize=12)
plt.ylabel('Number of Days', fontsize=12)
plt.xticks(days_count_418['year'], rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```